GemSort

An Automated Colour Based Gem Sorting System

Final Project Portfolio

Name	Balaji Rayudu S
Roll Number	BL.EN.U4EAC24011
Course	Electronic Systems Design and Automation (23EAC212)
Semester	IV (Even Semester, 2025–26)
Faculty Mentor	Dr. Sushant Shendre
Department	Electronics and Communication Engineering
Institution	Amrita Vishwa Vidyapeetham, School of Engineering, Bengaluru Campus

My role in the GemSort project is System Integrator and PCB Designer. As one of the system integrators on the team, I am responsible for designing the custom motherboard — from schematic capture to final PCB layout in EasyEDA — that serves as the central electronic controller for the sorting machine. The schematic is developed collaboratively with fellow system integrators, and the PCB layout is done with input from the broader team. My specific focus includes the power delivery architecture, motor driver interfacing, signal protocols, sensor integration, routing strategy, and EMI mitigation.

1. Project Overview

GemSort is an automated color-based gem sorting system built to identify and separate colored gems into designated bins without human intervention. The system is successfully developed as a team of 7 members, where each person handled a different subsystem.

The machine works in four sequential phases:

Phase 1 — Feeding & Singulation

A 60° angled feeding tube prevents jams and guides gems into a 1-hole rotating top disk, driven by a 12V 60 RPM BO motor. The single hole picks up one gem at a time from the tube, ensuring only one gem is ever in the system at once.

Phase 2 — On-Disk Color Scanning

As the top disk rotates approximately 90°, the gem (still sitting inside the hole) passes directly under a TCS34725 I2C color sensor. The ESP32 captures multiple raw RGB samples using a Peak Saturation tracking algorithm to lock onto the dead center of the gem while it is still on the disk.

Phase 3 — Drop & Transport

After another ~90° of rotation, the hole in the rotating top disk aligns with the single hole in the stationary bottom disk. The gem drops through onto a moving conveyor belt, driven by a 12V motor using a single-drive idler system.

Phase 4 — Sorting

The belt carries the gem to a 35° fixed ramp. The ESP32 recalls the color scanned on the disk, calculates the correct bin angle, and fires a PWM signal on GPIO 15 to an SG90 servo motor that snaps a 7-bin rotary carousel into position before the gem arrives.

The entire system relies on autonomous, continuous control loops powered by a raw C++ ESP32 firmware implementation, enabling high-speed optical math that OpenPLC could not support.

2. My Contribution

As one of the system integrators, I worked closely with my teammates to design the custom PCB motherboard that controls this entire system. The schematic was developed collaboratively among the system integrators — we discussed pin assignments, power topology, and component selection together before I captured the final design in EasyEDA. The PCB layout was also a team effort, with teammates providing feedback on component placement and mechanical fit with the sorting machine frame.

Specifically, my work covers:

- Schematic capture in EasyEDA (Rev 1.0) using net-label methodology, in collaboration with fellow system integrators
- Component selection and modular driver interfacing (TA6586 & DRV8833)
- GPIO pin assignment for the ESP32 DevKit (discussed and finalized as a team)
- Multi-voltage power distribution network design (12V / 5V / 3V / 3.3V)
- PCB layout with manual power routing and auto-routed logic signals
- Bottom-layer ground copper pour for noise mitigation

- Advanced software integration, including Peak Saturation tracking and HSV color space implementation
- Signal protocol management (I2C, PWM, and Digital Logic levels)
- Design Rule Check (DRC) validation and Gerber file generation

Note on physical implementation: The actual working prototype uses a combination of perfboard and breadboard as a hybrid wiring platform. The custom PCB is designed as a proper engineering exercise in EasyEDA — taking the circuit from schematic capture through complete layout, DRC validation, and Gerber generation — but is not fabricated due to manufacturing timelines.

3. Schematic Design Methodology

3.1 Modular Plug-and-Play Architecture

The first major design decision — discussed among the system integrators — is choosing a modular architecture over a monolithic one. We design the board to accept pre-built breakout modules through female header strips.

The control architecture uses the following core modules:

- **U1:** ESP32-DEVKITC (30-pin DevKit, mounted on dual-row headers)
- **U2:** TA6586 Motor Driver (5A continuous, handles the heavy 12V belt motor)
- **U3:** DRV8833 Motor Driver (handles the BO singulator motor)
- **U4:** LM2596 ADJ buck converter module (adjustable, tuned to 5V output)

3.2 Complete GPIO Pin Assignment Table

Every ESP32 GPIO is deliberately assigned based on pin capability and safety:

ESP32 Pin	Net Label	Connected To	Rationale
IO13	BELT_IN1	TA6586 IN1	PWM-capable, drives the HP Belt Motor forwards
IO14	BELT_IN2	TA6586 IN2	Drives the HP Belt Motor backwards (tied to logic low)
IO27	DISK_IN1	DRV8833 IN1	PWM-capable, drives the BO Singulator Motor
IO25	DISK_IN2	DRV8833 IN2	Drives the BO Singulator Motor backwards
IO15	SERVO_PWM	SG90 Servo Signal	PWM-capable, safe pin. Rotates the 7-bin carousel
IO34	IR_SENSE	IR Sensor 1	Input-only pin for digital beam break detection
IO22	I2C_SCL	TCS34725 SCL	Default I2C clock pin on ESP32
IO21	I2C_SDA	TCS34725 SDA	Default I2C data pin on ESP32

3.3 Net Label Strategy

The entire schematic is connected using net labels instead of direct wire lines. Every signal has a named label, and matching labels on different parts of the schematic are understood by EasyEDA as electrically connected. This keeps the schematic visually clean and prevents wrong connections.

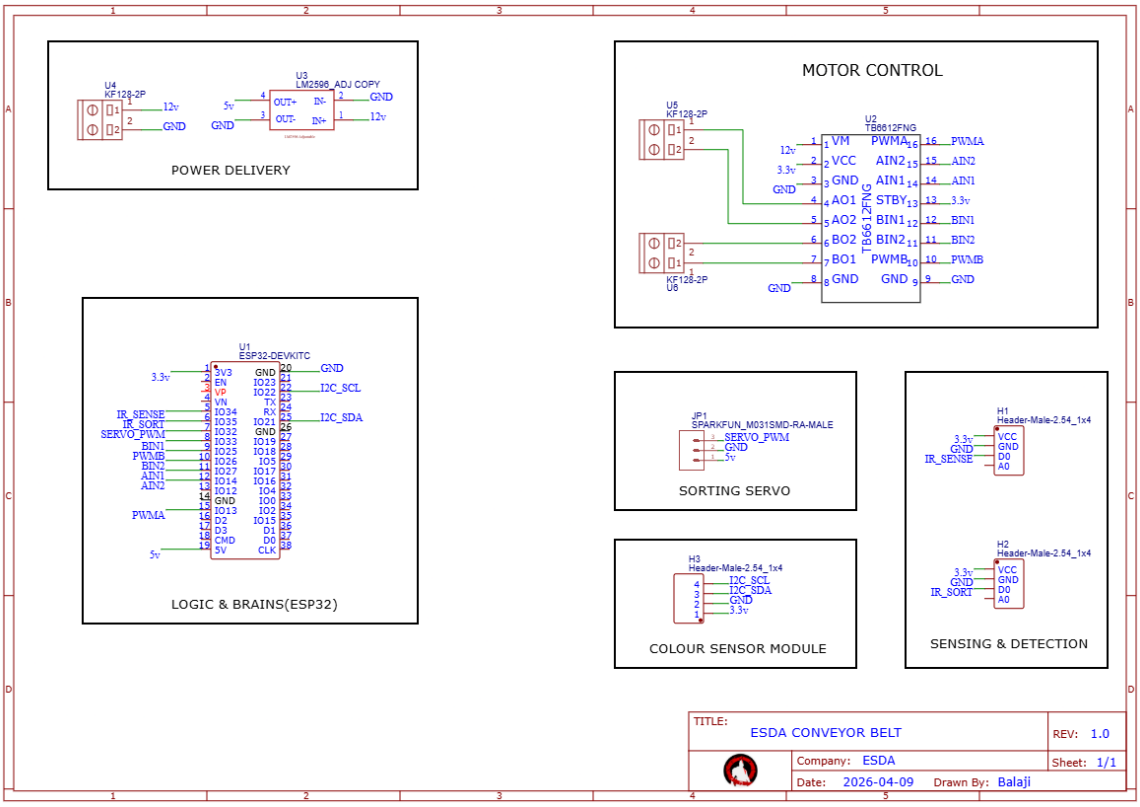
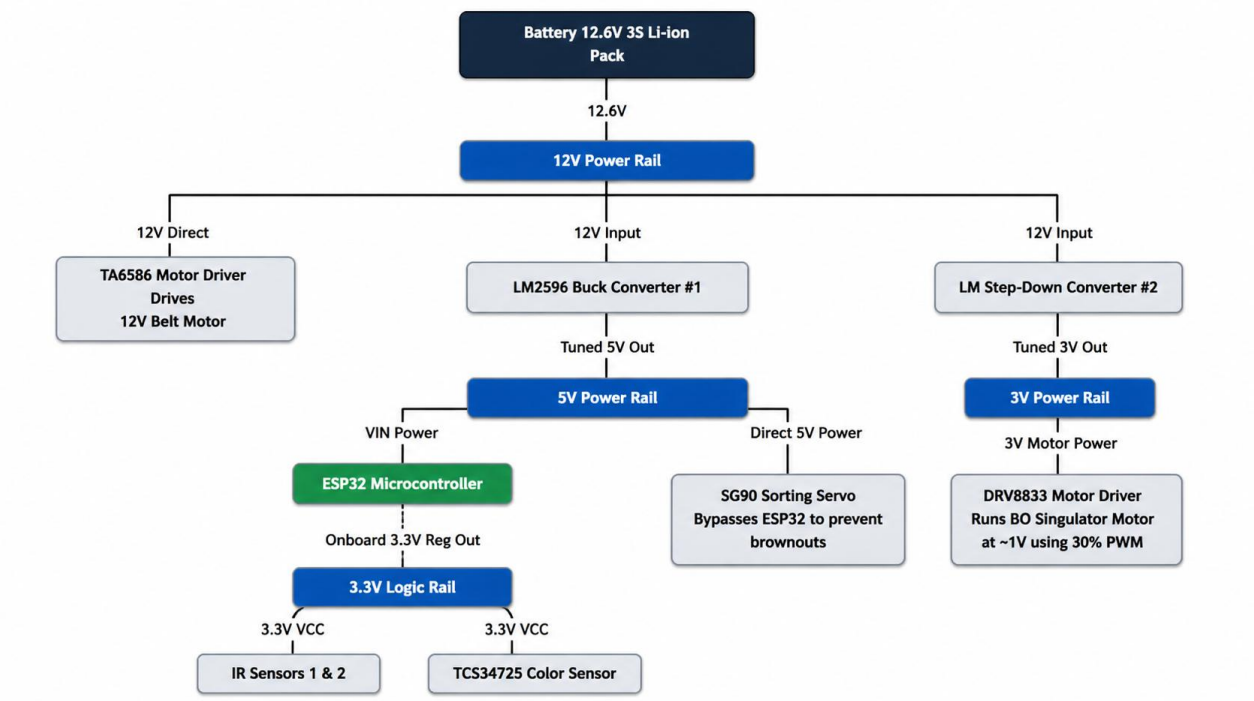


Figure 1: GemSort Initial Schematic — EasyEDA (Rev 1.0, 2026-04-09)

4. Power Distribution Network (PDN)

The power architecture is the most critical part of the design, because this system mixes high-current 12V DC motors with sensitive 3.3V I2C sensors on the same circuit.

4.1 Four-Rail Power Topology



4.2 Ultra-Low Speed Singulation (1V Virtual Rail)

A major challenge is driving the small BO motor at extremely low speeds (~18 RPM) for reliable singulation. I handle this by using a dedicated LM step-down converter to feed exactly 3V to the DRV8833 motor driver. Then, I apply software PWM limiting on the ESP32 (capping the duty cycle at ~30%). This reduces the average voltage delivered to the BO motor to roughly 1V, achieving the perfect slow, steady rotation required for the optical sensor to accurately scan the gems.

4.3 Why the Servo Gets Direct 5V from the LM2596

During breadboard testing, every time the servo snaps to a new position, the ESP32 resets and reboots. The servo draws a short burst of 1–2A when it moves under load. When powered through the ESP32's 5V output pin, that current spike pulls the ESP32's internal supply below its brownout detection threshold. In the final design, the servo's power line is routed directly to the LM2596's 5V output, which eliminates the brownout resets.

5. Software & Vision Engine

Initially, the machine was tested using OpenPLC to simulate industrial automation. However, due to the complexity of I2C color sensing and rapid positional servo math, the final master controller uses native C++ firmware on the ESP32.

5.1 Why Hue is Better than RGB

Raw RGB (Red, Green, Blue) values are extremely susceptible to ambient light changes and shadows. If a room gets darker, the raw R, G, and B numbers all drop drastically, making it impossible to use hardcoded RGB thresholds for sorting. To fix this, our firmware converts the raw RGB values into HSV (Hue, Saturation, Value) space. The Hue represents the actual color identity (e.g., Red is 0°, Blue is 240°) and remains mathematically constant regardless of how bright or dark the environment is.

5.2 Peak Saturation Tracking

Because the singulator disk hole is larger than the 13mm gems, the TCS34725 sensor often triggers on the edge of the hole, catching a shadow or the wood base beneath the gem. A simple fixed time delay fails because bright gems and dark gems trigger the optical threshold at different times. I developed a continuous sampling loop: when the sensor triggers, it rapidly takes 6 readings over 250ms and saves the RGB values with the highest Saturation. Because brightly colored gems always have a higher saturation than dark wood or the disk's matte black paper, this mathematically guarantees the sensor perfectly locks onto the 'dead center' of the gem.

5.3 White Spot Anomaly Filter

A small, highly reflective scratch on the matte black singulator disk was occasionally triggering the sensor and being misclassified. By analyzing the raw data, we found the white spot has a unique signature: a Hue of 15°–30° but a low Saturation (<55%). A mathematical trap is explicitly coded to ignore this signature, allowing the machine to spin autonomously without false servo actuations.

6. PCB Layout and Routing Strategy

The PCB layout follows a deliberate zoned placement strategy that physically separates high-power motor circuits from sensitive sensor circuits. This is a critical EMI mitigation technique to prevent the brushed DC motors from corrupting the I2C color sensor data.

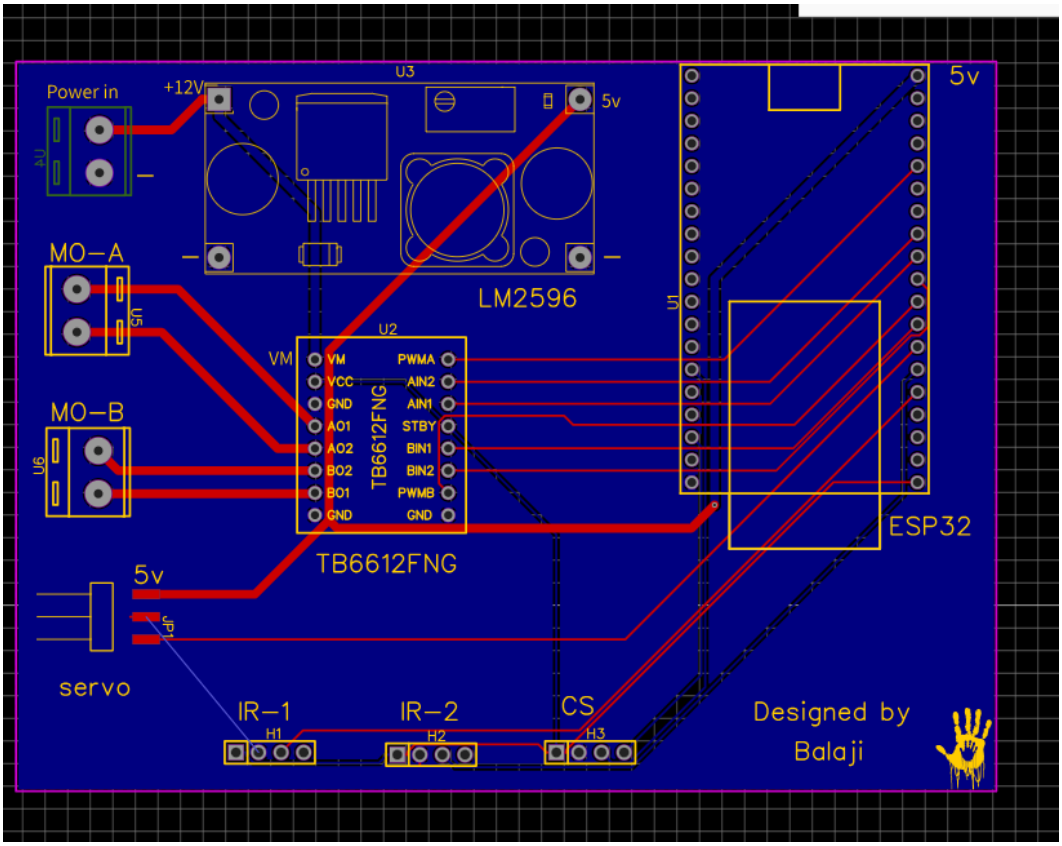


Figure 2: GemSort PCB Layout — EasyEDA

6.1 Bottom Layer Ground Pour

The entire bottom copper layer is flooded with a solid copper pour connected to the GND net. This serves three purposes:

- **Low-impedance ground return path —** Every component's ground pin connects to the battery ground through the copper plane directly beneath it, instead of through a single thin trace.
- **Thermal relief —** The motor drivers and buck converter generate heat during operation. The copper area spreads that heat across the board.
- **EMI shielding —** The ground plane reduces current loop area for all return paths. By Faraday's law, radiated electromagnetic field strength is proportional to the area of the current loop. The ground pour collapses these loops to near-zero area.

7. Challenges and Solutions

S.NO	Challenge	Root Cause	Solution
1	TB6612FNG Driver Burnout	Motor stall current (>1.2A) exceeded chip thermal limits.	Upgraded to heavy-duty TA6586 (5A) for the belt, and DRV8833 for the singulator.
2	Mechanical Jamming at Hopper	Vertical gem stacking created intense downward pressure on the disk, stalling the BO motor.	Replaced the vertical tube with a 60° angled tube to relieve direct downward gravity.
3	Drivetrain Fighting	Mismatched worm gear vs straight gear caused parasitic drag on the conveyor belt.	Switched to a single-drive idler system, eliminating the mechanical bind.
4	Color Shifting & Edge Errors	Sensor triggered on the shadow/wood edge of the singulator hole before the gem arrived.	Replaced basic thresholding with the Peak Saturation tracking algorithm in HSV space.
5	Servo Brownout	Servo 1–2A inrush current pulled supply below ESP32 brownout threshold.	Routed servo power directly from LM2596 5V output; ESP32 only sends PWM signal.

8. Signal Analysis & Communication Protocols

The GemSort motherboard relies on three distinct types of electrical signals to facilitate communication between the ESP32 brain, sensors, and actuators. These protocols were deliberately chosen to maximize noise immunity in an electrically noisy (motor-heavy) environment.

8.1 I2C (Inter-Integrated Circuit)

Used for: TCS34725 Color Sensor.

- Digital Protocol
- 3.3V Logic Level (Amplitude)
- Standard Mode (100 kHz) to Fast Mode (400 kHz)
- 2 Wires: SDA (Serial Data) and SCL (Serial Clock) + VCC/GND

Why I2C? Color sensing requires transmitting complex 16-bit data arrays (Red, Green, Blue, Clear). Sending this data as an analog voltage would be disastrous because the magnetic noise from the spinning DC motors would corrupt the amplitude, changing the perceived color. I2C transmits this data digitally using binary 1s and 0s over a synchronized clock, ensuring perfect mathematical data integrity regardless of nearby motor noise.

8.2 PWM (Pulse Width Modulation)

Used for: SG90 Servo Motor and DRV8833/TA6586 Motor Speed Control.

- Digital Square Wave
- 3.3V Amplitude (from ESP32 GPIOs)
- 50 Hz (20ms period) for the Servo. High frequency (5kHz–20kHz) for the DC motors.

Why PWM? Instead of varying the actual analog voltage to change motor speed or servo position (which causes massive heat loss and torque reduction), PWM pulses a full 3.3V digital signal rapidly. For the servo, a 1ms pulse width tells it to go to 0°, and a 2ms pulse tells it to go to 180°. For the BO singulator motor, we limit the PWM duty cycle to 30%, which simulates a 1V average supply without actually dropping the peak voltage, preserving rotational torque at ultra-low speeds.

8.3 Simple Digital Interrupts (GPIO Logic)

Used for: IR Proximity Sensors.

- Discrete Digital Logic
- 3.3V (HIGH) or 0V (LOW) Amplitude
- Asynchronous (Event-Driven)

Why Digital Logic? The IR sensors feature onboard comparator ICs (LM393) that handle the analog-to-digital conversion directly at the sensor. When a gem breaks the beam, the sensor snaps its output cleanly from HIGH (3.3V) to LOW (0V). The ESP32 simply waits for this falling edge interrupt. This frees the ESP32 from having to continuously poll an analog pin, saving massive amounts of processing power for the HSV color math.

9. Course Outcome Mapping

CO1: Understand the Physical Architectural Design Fundamentals for Electronic Systems

This project demonstrates physical architectural design through:

- **Modular architecture decision —** Choosing plug-and-play module headers (ESP32 DevKit, Motor Drivers, LM2596 modules) over bare ICs, with explicit trade-off analysis of board size vs repairability vs design complexity.
- **Component placement strategy —** Placing components based on signal flow and current path optimization. Power input and buck converters in the top-left, motor drivers on the left edge, ESP32 centrally on the right, sensors along the bottom.
- **Multi-voltage power topology —** Designing a four-rail PDN (12V direct → 5V regulated → 3V regulated → 3.3V from ESP32) where each voltage level serves specific components based on their operating requirements.
- **Physical form factor —** Board dimensions, screw terminal placement on outer edges for wire access, and header orientation matching the mechanical frame of the sorting machine.

CO2: Examine the Impact of Interconnections on Electronic System Performance at Various Abstraction Levels

The GemSort PCB design demonstrates interconnection impact at three abstraction levels:

Schematic Level (Logical Interconnections)

- Net-label methodology ensures every connection is named and traceable (SERVO_PWM, I2C_SDA, DISK_IN1, etc.).
- Logical grouping into labeled bounding boxes shows system partitioning at design time.

PCB Level (Physical Interconnections)

- Trace width directly affects current capacity. The 1.0mm power traces carry heavy motor current safely; a 0.254mm trace carrying the same current would overheat.
- Trace length on I2C lines affects signal integrity. Longer traces add parasitic capacitance, which slows clock edges and can cause I2C communication failures.

System Level (Subsystem Interconnections)

- The timing between IR Sensor 1 (scan trigger) and IR Sensor 2 (sort trigger) relies on precise physical distance on the conveyor belt matching the firmware's tracking array.
- The servo's actuation accuracy depends on clean PWM signal delivery. Direct 5V power routing to the servo prevents voltage drops that cause brownouts.

CO3: Understand EMC & EMI Issues in Electronic Systems Design

The GemSort system is a practical EMI scenario: two brushed DC motors (broadband noise sources) operating on the same circuit as a 3.3V I2C color sensor and a microcontroller with GPIO-level logic.

Technique	Implementation in PCB Design	EMC Effect
Zoned physical separation	Motors/driver in top-left; sensors in bottom-right	Reduces near-field coupling (coupling $\propto 1/d^2$)
Solid ground copper pour	Bottom layer flooded with GND net	Reduces return current loop area → reduces radiated emissions
Separated trace routing	Power traces along top/left; signal traces along bottom/right	Prevents parallel conductor coupling
Dedicated servo power	Direct LM2596 → Servo trace, bypassing ESP32	Eliminates supply-coupled noise from servo inrush
Trace width sizing	1.0mm for power vs 0.254mm for signals	Prevents voltage drops causing ground bounce

CO4: Understand and Develop the Frameworks Required for Automation

The GemSort system implements a highly advanced automation framework using the ESP32 as a C++ embedded controller, far surpassing basic PLC capabilities:

- **Peak Saturation Tracking —** Rather than relying on simple delays or basic thresholding, the firmware samples the hole multiple times and calculates the Peak Saturation (HSV space) to lock onto the dead center of the gem, actively filtering out shadows and disk anomalies.
- **Event-Driven State Machine —** The sorting logic follows a deterministic state machine triggered purely by physical interrupts. The IR sensor falling-edge triggers the scanner, preventing unnecessary polling.
- **Continuous Conveyor Operation —** The machine successfully singulates, spaces, scans, and sorts continuous streams of objects autonomously, mirroring industrial packaging plants.

10. PCB Design Certification

Completed the Altium Education PCB Design course, covering schematic capture fundamentals, component library management, PCB layout best practices, design rule configuration, and Gerber file generation.

Altium Designer Education — Certificate of Completion

11. Design Screenshots

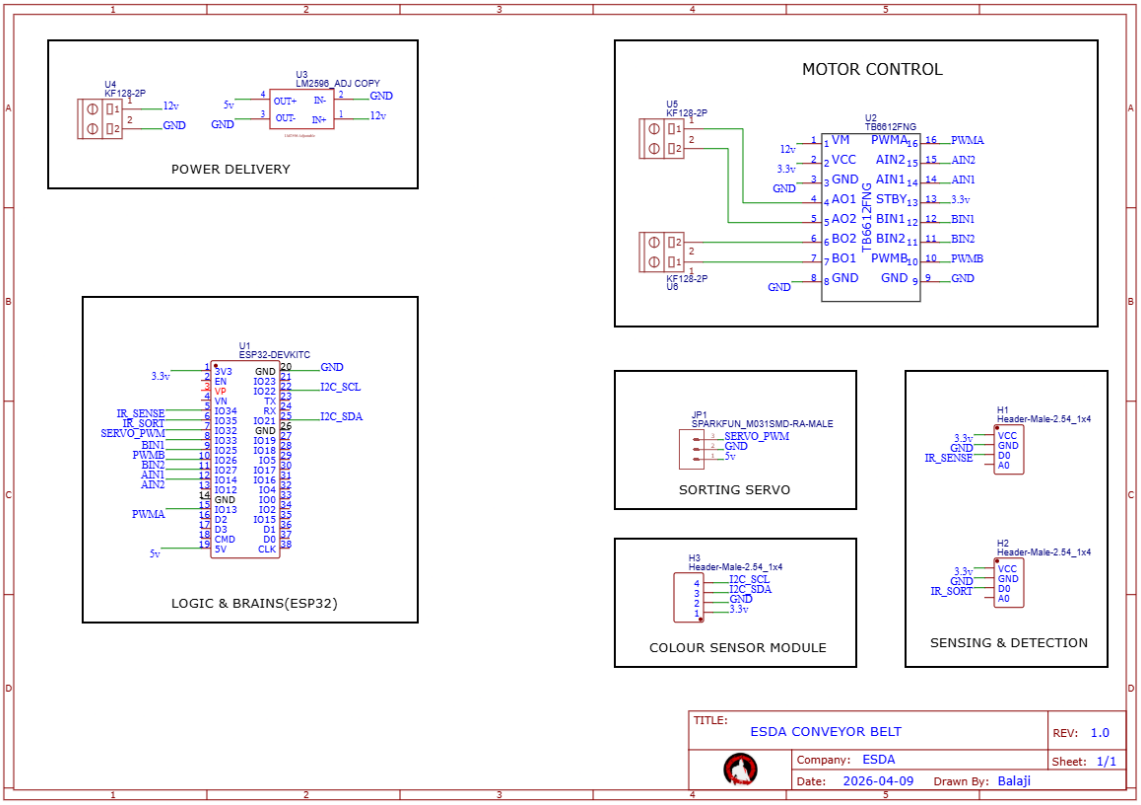


Figure 3: Complete Schematic with labeled bounding boxes and net labels

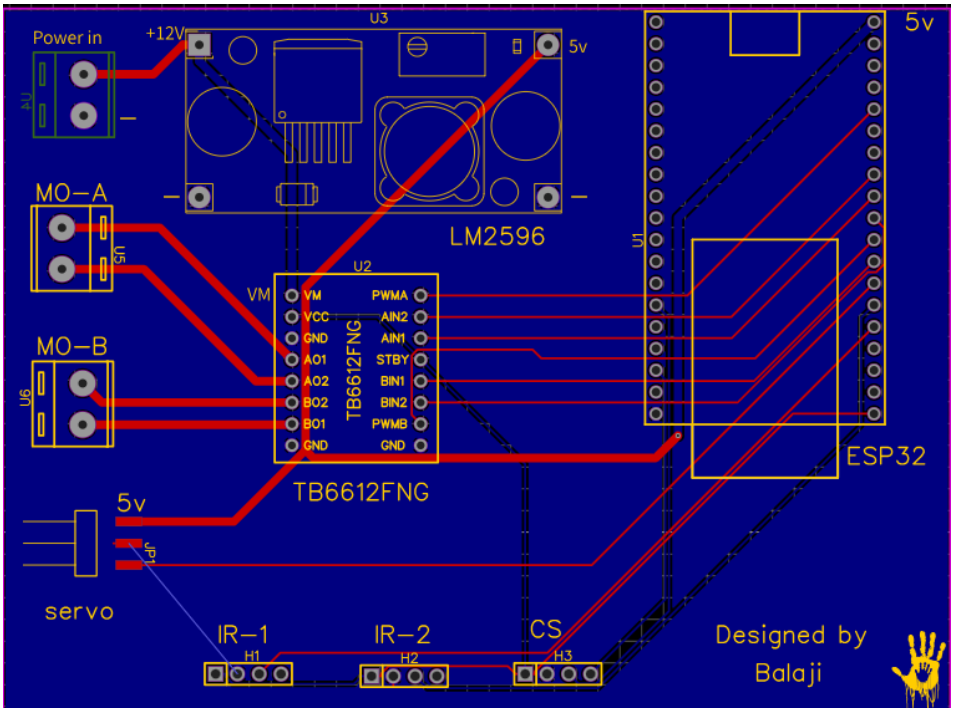


Figure 4: PCB Layout showing zoned component placement and power trace routing

12. Key Learnings

Working on the PCB design, firmware, and physical prototype taught me several things that textbook theory alone does not communicate:

- **Mechanical Theory vs Physical Reality:** The original vertical feeding tube placed the entire weight of the gem stack directly onto the separation disk, stalling the motor. Changing to a 60° angled tube relieved the gravity and completely fixed the jamming. Mechanical physics dictate electronic performance.
- **Drivetrain Friction is a Killer:** Using a mismatched worm gear and straight gear caused massive parasitic drag. Simplifying to a single-drive idler system allowed the conveyor to run flawlessly.
- **Hue is superior to RGB for Machine Vision:** Raw RGB values change constantly based on room shadows. Converting raw sensor data into HSV (Hue, Saturation, Value) space allowed the machine to sort based on true color identity, gaining immunity to ambient light.
- **Ground planes are engineering tools, not shortcuts:** The bottom-layer ground pour is the most electrically significant feature on the board design. Understanding its role in EMI reduction changed how I think about grounding even on the perfboard prototype.
- **Physical layout is an engineering decision:** Where you place components matters as much as how you connect them — a lesson we applied to both the PCB design and the perfboard wiring to prevent motor noise from corrupting I2C data.

13. Working Demonstration

The following section provides a visual walkthrough of the completed GemSort machine in operation. Each stage of the autonomous sorting pipeline is shown below.

Stage 1 — Feeding & Singulation

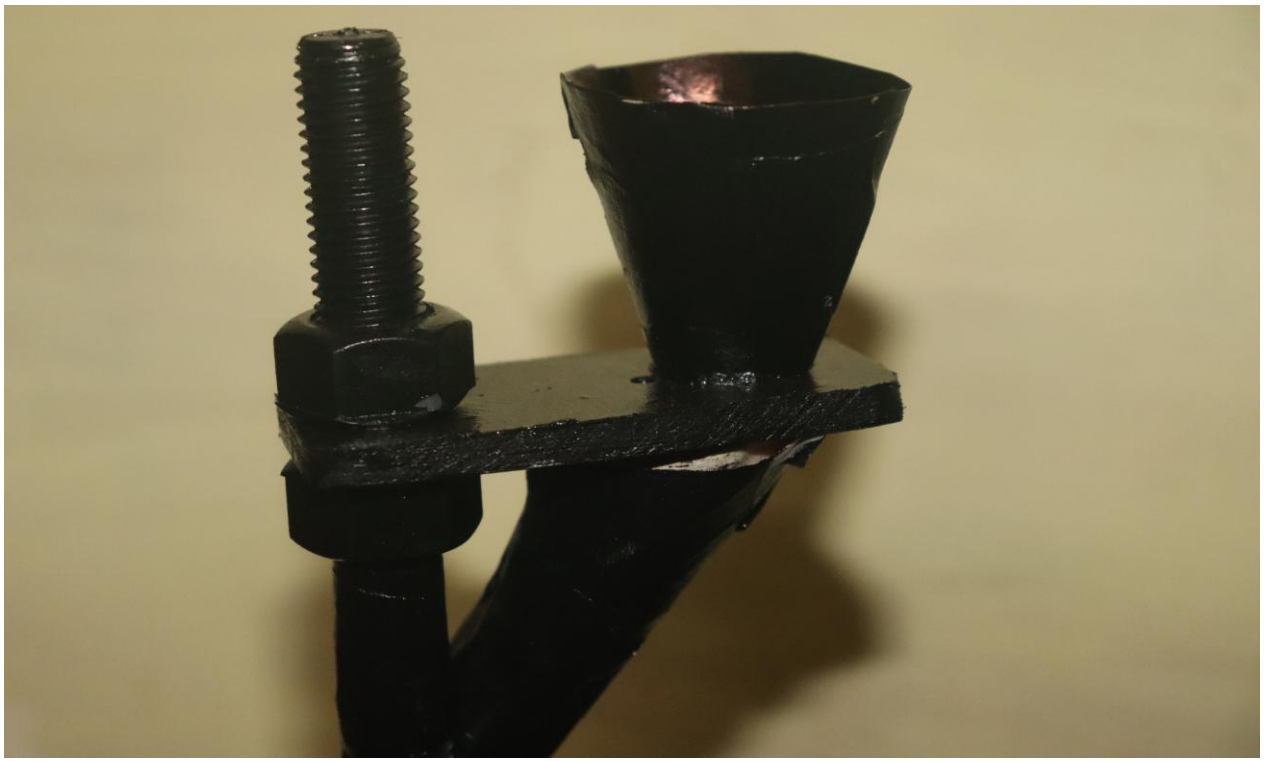


Figure 3: The 60° angled feeding tube and 1-hole singulator disk

Stage 2 — On-Disk Color Scanning

After the disk rotates approximately 90°, the gem — still sitting inside the hole — arrives directly under the TCS34725 color sensor. The ESP32 rapidly takes 6 readings over 250ms and selects the one with the highest color saturation. This 'Peak Saturation Tracking' algorithm guarantees the sensor always reads the dead center of the gem while it is still on the disk.

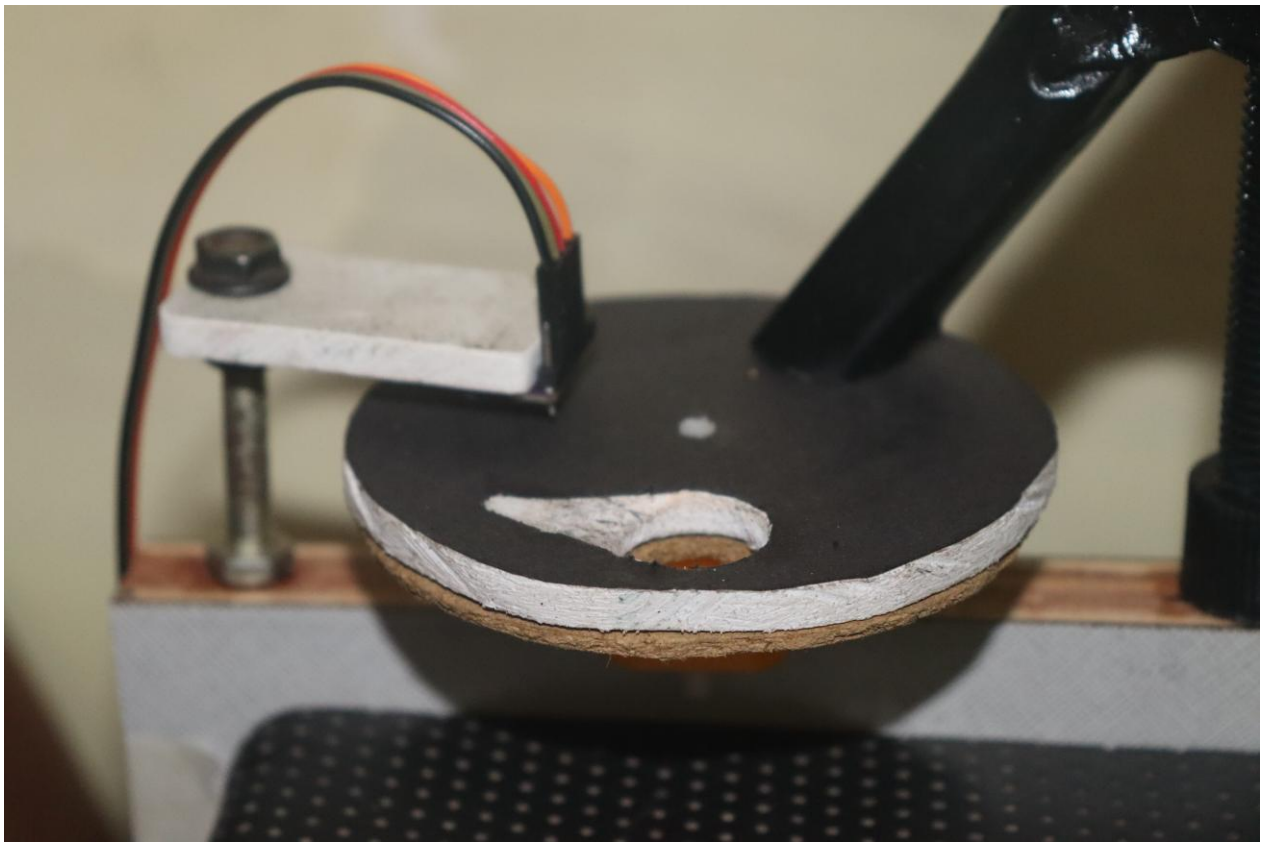


Figure 4: TCS34725 color sensor scanning a gem through the singulator hole

Stage 3 — Drop & Transport

After another ~90° of rotation, the hole in the rotating top disk aligns with the single hole in the stationary bottom disk. The gem drops through onto the conveyor belt, which is driven by a 12V HP Printer motor using a single-drive idler system. The belt transports the gem toward the sorting ramp.

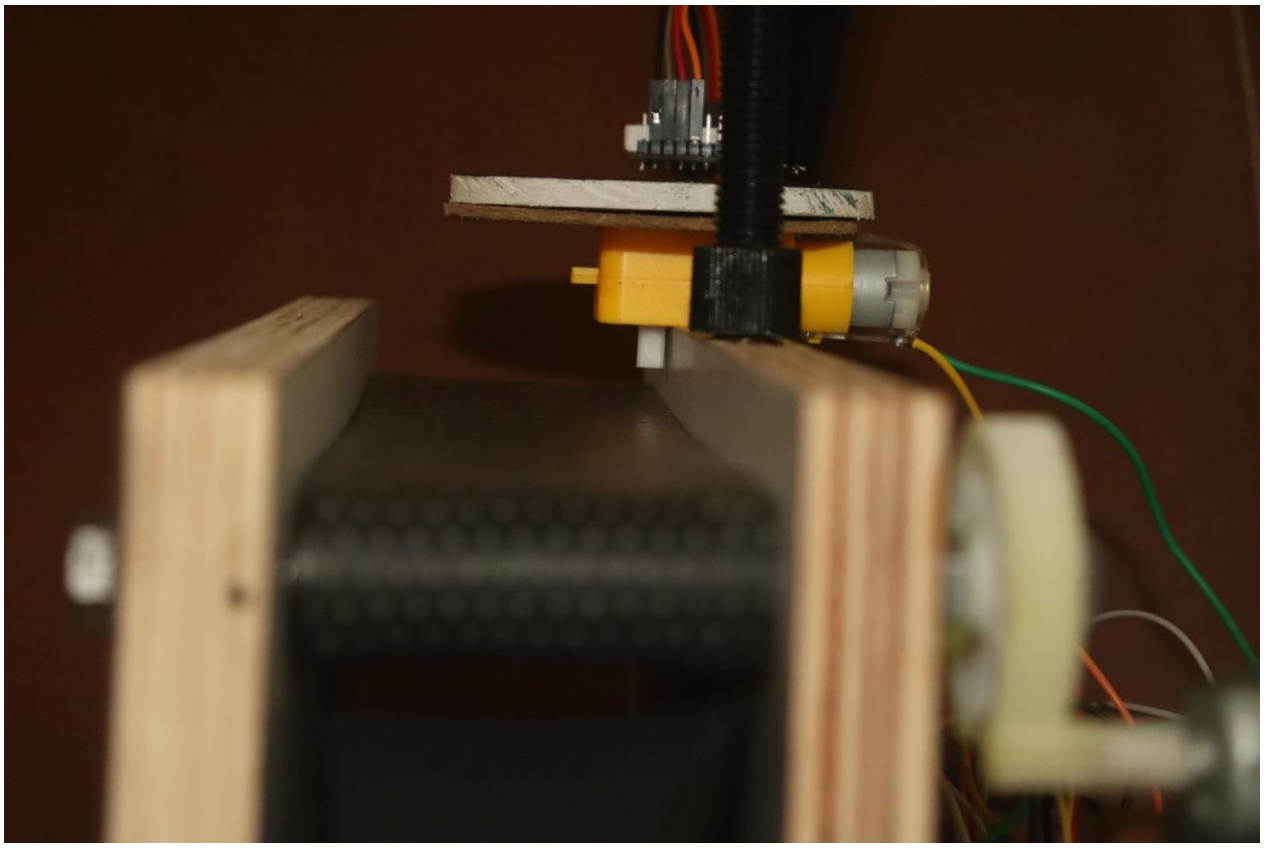


Figure 5: Gem dropping from bottom disk onto conveyor belt

Stage 4 — Sorting into Bins

At the end of the belt, the gem slides down a 35° ramp into a 7-bin rotating carousel. Based on the Hue value scanned earlier on the disk, the ESP32 commands the SG90 servo to rotate the carousel to the correct bin position before the gem arrives.

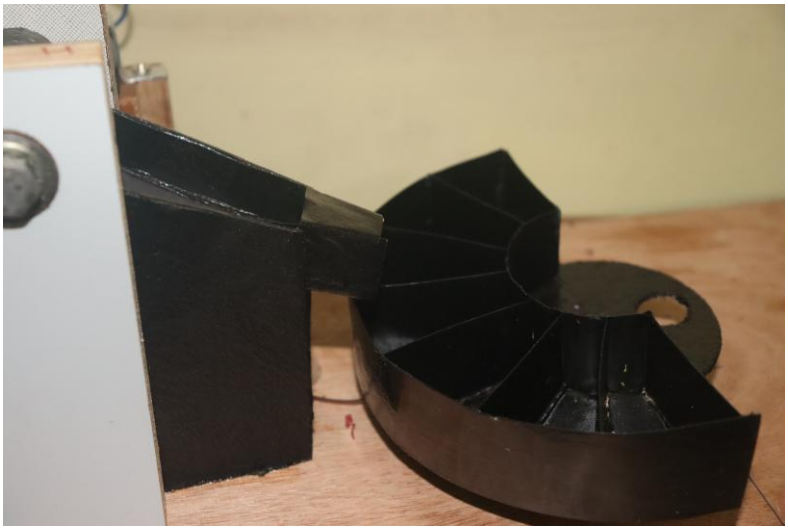


Figure 6: The 7-bin carousel with gems sorted by color

14. Sorting Results & Video Demonstration

The following images show the final sorted output after a full autonomous run. Each bin contains only gems of a single color, demonstrating the accuracy of the HSV-based classification engine and the Peak Saturation tracking algorithm.

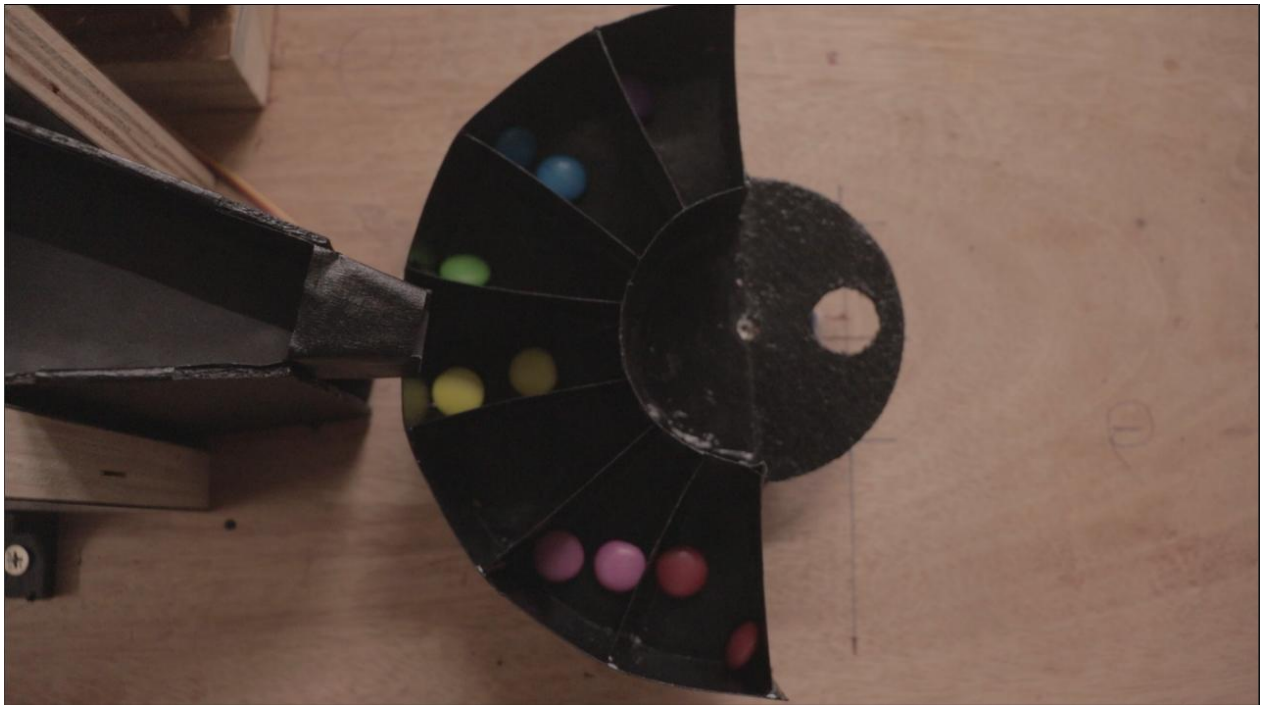


Figure 7: Top view of all 7 bins after autonomous sorting

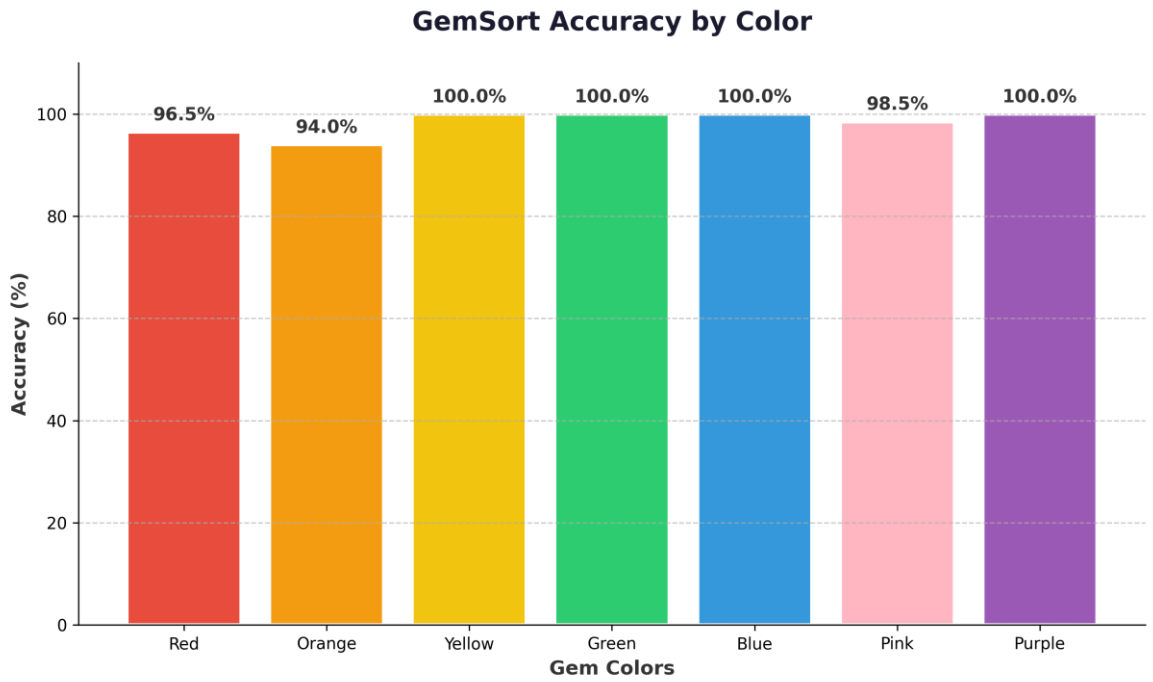


Figure 7.1: Final Sorting Accuracy by Color

A full video demonstration of the GemSort machine performing autonomous color sorting is available at the link below:

Working Demonstration

Click here to view the Working Demonstration Video

15. Conclusion

GemSort is a successfully completed, fully autonomous gem sorting machine. As a system integrator, my role required looking after all levels of the project: from the mechanical drivetrain and motor tuning to the electrical wiring, sensor calibration, and advanced C++ vision firmware.

The problems we solved were not theoretical. The servo brownout, the motor noise corrupting I2C readings, the physical weight of the gems stalling the singulator motor, and the optical interference from shadows — these all happened on our workbench. We successfully resolved them one by one by understanding the underlying electronics, applying proper PCB design practices, and writing advanced tracking algorithms.

This project directly demonstrates physical architecture mastery (CO1), interconnection impact at multiple abstraction levels (CO2), EMC/EMI-aware design with a documented case study (CO3), and highly advanced event-driven automation frameworks (CO4). The final GemSort machine stands as a complete, robust real-world implementation of the Electronic Systems Design and Automation curriculum.